

Android Low Memory Killer (LMK) Shielding & Daemon Service Lifecycle Specification

1. Problem Statement: Android LMK vs. Edge AI Daemons

The Android Linux kernel employs the **Low Memory Killer (LMK)** daemon to reclaim RAM when available memory drops below system watermarks. Standard background threads and un-promoted Android Services receive high Out-Of-Memory adjustment scores (`oom_score_adj` up to 1000). When Qwen 3.5 9B (~5GB footprint) allocates tensor memory, the kernel will aggressively terminate background processes, terminating the user's AI session.

To ensure continuous, headless execution of the `Asc` Terminal Daemon, `Ayre` Media Daemon, and NPU model switchers, the system implements a multi-tier **LMK Shielding Protocol**.

2. Process Prioritization & LMK Adjustment Tiers

Android manages process eviction order using `oom_score_adj` (ranging from -1000 to +1000):

Process Category	<code>oom_score_adj</code> Target	LMK Eviction Vulnerability	System Action
System Native Services	-1000	Immune	Core Android init / hardware HALs
Foreground Service (Audio / Assistant)	-16 (FOREGROUND_APP_ADJ)	Highly Protected	Active notification + Foreground Service Type
Visible UI (Horizons WebView)	0	Protected	Currently focused application screen
Perceptible Background (Accessibility)	100-200	Shielded	Android Accessibility / Assist Service role
Standard Cached Process	900-1000	First to be Killed	Default background service state

3. Implementation of the LMK Shielding Protocol

1. Persistent Foreground Service Notification (**FOREGROUND_APP_ADJ**)

In Android 14/15, foreground services require explicitly declared runtime types.

AescDaemonOrchestrator.kt and **Æyre** declare:

```
// AndroidManifest.xml
```

```
<service  
  
    android:name=".services.AescDaemonOrchestrator"  
  
    android:foregroundServiceType="specialUse|connectedDevice"  
  
    android:exported="false" />
```

In Kotlin:

```
val notification = NotificationCompat.Builder(this, CHANNEL_ID)  
  
    .setContentTitle("NovÆxopia Sovereign Daemon Active")  
  
    .setContentText("Local ADB loopback :5555 & NPU socket online")  
  
    .setSmallIcon(R.drawable.ic_daemon_shield)  
  
    .setOngoing(true)  
  
    .setPriority(NotificationCompat.PRIORITY_MIN) // Silent but persistent  
  
    .build()  
  
startForeground(NOTIFICATION_ID, notification)
```

2. Service Restart Policy (**START_STICKY**)

If an extreme memory spike forces the kernel to reclaim the daemon, **onStartCommand** returns **START_STICKY**:

```
override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
```

```

super.onStartCommand(intent, flags, startId)

ensureDaemonsSupervised()

return START_STICKY // Instructs OS to recreate service when RAM is freed
}

```

3. CPU Power Headroom: Partial Wake-Locks

To prevent the AP (Application Processor) from entering deep sleep while an agent executes long-running background reasoning or code review loops:

```

val powerManager = getSystemService(Context.POWER_SERVICE) as PowerManager

val wakeLock = powerManager.newWakeLock(

    PowerManager.PARTIAL_WAKE_LOCK,

    "novaecopia:aesc_npu_wakelock"

)

wakeLock.acquire(10 * 60 * 1000L) // 10 minute bounded lease

```

4. Accessibility Service & Assistant Privilege Escalation

By registering [Aesc](#) as an **Android Accessibility Service** and designated **Default Assistant App**:

- Grants the daemon persistent operating life without aggressive battery optimization throttling.
- Bypasses background process limits on Android 14/15.
- Grants elevated UI event observation and hardware key interception without requiring root.